

Agent acceleration

Making LLM agents that operate software finish a task in less time and for less money, without finishing fewer tasks.

Computer-use, web and coding agents, which work a desktop, a browser or a code base one step at a time through a model.

September 2026

PART 1

Where the time and the money go

Why LLM agents that operate software are slow and expensive, and how the two are linked.

Every number comes from a published measurement of a computer-use, web or coding agent, or from a vendor’s own price page (2024–2026); the conditions of each measurement are stated next to it, and the full references are at the end of the part.

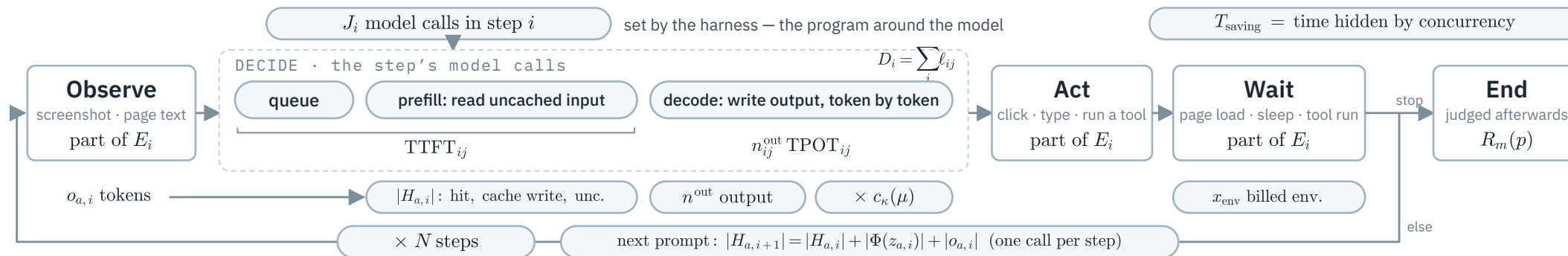
IN THIS PART

02	One attempt of an agent, step by step: the loop and its variables
03–04	The problem, defined: time, the growing prompt, money, success and the goal
05–10	Where the time goes, term by term
11–12	Where the money goes, term by term, and per success
13–14	How time and money are linked, and the levers
15–16	References · Appendix A0–A5

One attempt of an agent, step by step

here: the loop one attempt runs, and the variable at each step → to: the formulas built from them

One attempt is N steps; each step is some model calls plus everything that is not a model call. Every variable below is defined here once; pages 3–4 add them up.



i , N step (one observe–decide–act–wait round); steps in the attempt

TTFIT_{ij} time to the first generated token: queueing, sending, prefill of the uncached input

T_{saving} time hidden because some of it ran at the same time (AsyncFC's term, intervals widened); 0 if strictly serial

n^{hit} , n^{w} , n^{unc} input tokens read from the cache, stored in it (a cache write, 5-min or 1-h: $n^{\text{w}} = n^{\text{w5m}} + n^{\text{w1h}}$), or uncached

$R_m(p)$ probability that one attempt of design m on task p succeeds

J_i , j model calls in step i (planner, judge, retries ...); call index

n_{ij}^{out} , TPOT_{ij} output tokens of the call, thinking included; time per output token after the first

$o_{a,i}$ tokens the result or screenshot of step i adds to the prompt of agent a

$c_\kappa(\mu)$, x_{env} price per token of class κ on serving model μ ; billed environment usage (e.g. sandbox hours)

ℓ_{ij} latency of call j of step i

D_i , E_i model time of step i ; all its other time (observe, act, wait, harness gaps, back-off)

$|H_{a,i}|$, $\Phi(z_{a,i})$ length in tokens of the prompt that step i 's call reads (one call per step); the call's output as written into that history

reading, writing reading = prefill of input tokens; writing = decoding output tokens (a cache write is storage, not output)

The problem, defined (1/2): the time of one attempt, and the growing prompt

from the loop → here: time as a sum over steps and calls, and why the prompt grows → to: money, success and the goal

each row says whether it is **verbatim** from its source, **adapted** (reason in Appendix A0) or **our distillation** (no source writes it as a formula) · calc. = our algebra

TIME ONE ATTEMPT adapted · our distillation	$T_{\text{attempt}} = \sum_{i=0}^N (D_i + E_i) - T_{\text{saving}}, \quad D_i = \sum_{j=1}^{J_i} \ell_{ij}$ $\ell_{ij} \approx \text{TFT}_{ij} + n_{ij}^{\text{out}} \text{TPOT}_{ij}$ $T_{\text{saving}} \triangleq \mathcal{S}(\mathcal{M}) + \mathcal{S}(\mathcal{E}) - \mathcal{D}(\mathcal{M} \cup \mathcal{E})$	T_{attempt} : wall-clock of one attempt, from hand-over to result. Model and non-model time add over the steps ($i=0$ holds calls outside any step). $\triangleq$: defined as; $\mathcal{S}$: summed length of intervals; $\mathcal{D}$: length of their union (not the model time D_i); $\mathcal{M}, \mathcal{E}$: model-call and non-model intervals of the trace. sum: Chen et al., 2026 · calls within a step: our distillation, evidence Abhyankar, Qi & Zhang, 2026 · call: Artificial Analysis, 2026 (third-party) · saving: Feng et al., 2026, intervals widened
PROMPT GROWS adapted · our distillation (calc.)	$ H_{a,i+1} = H_{a,i} + \Phi(z_{a,i}) + o_{a,i} $ $\sum_{i=1}^N H_{a,i} = N H_{a,1} + \frac{N(N-1)}{2} \bar{g}$	Each step appends its output and its result to the next prompt (one call per step, full history kept). $\bar{g}$: the added tokens per step, weighted by how often they are re-read. The N^2 term dominates beyond $N^* = 2 H_{a,1} /\bar{g} + 1$ (calc.). recurrence: Yuan et al., 2026 (lengths of its Eq. 3) · sum: our distillation (calc.) · re-sent history, measured: Abhyankar, Qi & Zhang, 2026

The problem, defined (2/2): the money of one attempt, per success, and the goal

from time and the prompt → here: money, cost and time per success, and what counts as acceleration → to: where the time goes

verbatim · adapted · our distillation, as on page 3 · calc. = our algebra

MONEY
ONE ATTEMPT
adapted

$$c_m(p) = \sum_{i=0}^N \sum_{j=1}^{J_i} \sum_{\kappa \in K} n_{ij}^\kappa c_\kappa(\mu_{ij}) + x_{\text{env}} c_{\text{env}}$$
$$K = \{\text{hit, w5m, w1h, unc, out}\}$$

SUCCESS
PER SUCCESS
verbatim ·
adapted

$$v(m, p) = \frac{C_m(p)}{R_m(p)}, \quad \frac{\mathbb{E}[T_{\text{attempt}}]}{R_m(p)}$$

GOAL
ACCELERATION
our
distillation

$$\text{Pareto}_m \left(\frac{\mathbb{E}[T_{\text{attempt}}]}{R_m(p)}, v(m, p) \right) \quad \text{s. t.} \quad R_m(p) \geq R_0$$

$c_m(p)$: dollars of one attempt of design m (model and harness) on task p . n_{ij}^κ : tokens of class κ in call j of step i — cache hit, 5-min or 1-h cache write, uncached input, output. c_{env} : price per unit of billed environment usage.
base: Erol et al., 2026, Eq. 13, extended as its App. D.1 allows · classes: Anthropic, 2026b · three-class precedent: Xu et al., 2026

$v(m, p)$: expected dollars per success; the second term: expected seconds per success. $C_m(p)$: expected $c_m(p)$ over all attempts, failures included; $\mathbb{E}$: expectation. Independent attempts, retried one after another until a verified success ($1/R_m(p)$ on average); the verifier’s time and cost count in every attempt.
money: Erol et al., 2026, Eq. 2 (verbatim) · time: same derivation in seconds, as its App. D.1 allows (adapted)

Among designs that succeed at least R_0 of the time, keep those no other design beats on both time and money per success; the reference is a person doing the task today. Per task p ; over a task mix,
 $\sum_p C_m(p) / \sum_p R_m(p)$.
distilled from Kapoor et al., 2025 (“jointly optimizing the two metrics”, arXiv v1) and Erol et al., 2026 (App. C.8: leave unreliable systems off the frontier)

WHAT THE FORMULAS DO NOT SHOW: THE VARIABLES ARE COUPLED

- The prompt length drives both sides: prefill time and the input bill (Abhyankar, Qi & Zhang, 2026)
- The harness and provider, not the model, set the cache split (Zhu et al., 2026)
- Time can cost money: a cache entry expires after 5 min or 1 h, so a slow step can turn cache reads into cache writes (our reading of Anthropic, 2026b)
- Thinking lengthens D_i and the output bill; calls and thinking change $R_m(p)$ (page 13)

Where the time goes: five classes, one per term

from the definition → here: the terms of the time of one attempt, each with one measurement → to: steps and calls

Four of the five are terms of the time equation; the fifth is how they combine: the concurrency term T_{saving} is small in the one production trace that reports it, so the terms nearly add. Which class dominates depends on the harness, not on the kind of agent (page 10). Cause-by-cause tables with measurement conditions: Appendix A1.

$$T_{\text{attempt}} \approx \left[\sum_i \left[\sum_{j \leq J_i} \left(\text{TTFT}_{ij} + n_{ij}^{\text{out}} \text{TPOT}_{ij} \right) + E_i \right] - T_{\text{saving}} \right]$$

I · TOO MANY STEPS (N)
Hour-scale tasks take hundreds of steps, and every step is a full round trip

- 318 tool calls per task on OSWorld 2.0 — 108 desktop tasks, Claude Opus 4.7, one action per step

(XLANG Lab, 2026)

II · TOO MANY CALLS PER STEP (J_i)
A harness that plans, judges and reflects multiplies every step

- 4–12 planning calls per judging call — GTA1 harness: four parallel planners, up to three retries, one judge

(Abhyankar, Qi & Zhang, 2026)

III · EACH CALL IS SLOW (ℓ_{ij})
Inside a call, waiting, reading and writing are three separate delays

- Writing, one token at a time, is 91–98.6% of model time — local 27–31B models with a warm cache

(Yuan et al., 2026)

IV · NON-MODEL TIME (E_i)
The environment costs time whether or not the model is running

- 6.6 s per browser action vs 4.7 s per model call, median — 151 WebVoyager live-site tasks, GPT-4o

(Wong et al., 2026)

V · LITTLE RUNS CONCURRENTLY (T_{saving})
Almost nothing overlaps, so task time is close to a sum: fixing one term saves only its own share

- Concurrency within a turn: 1.15 — GitHub Copilot production telemetry

(Liu et al., 2026)

I–II · Too many steps, too many calls per step

from the five classes → here: the two sums, steps and calls per step → to: what happens inside a call

N multiplies every term, and J_i multiplies N . Steps accumulate because tasks are long, each step does one action, navigation-only steps still go through the model, and the agent idles or loops; calls accumulate because the harness plans, judges and reflects, or samples several candidates per step.

I-1, I-2 · LONG TASKS, ONE ACTION PER STEP

Long tasks with one action per step need hundreds of steps; several actions per call cut them by half or more

- 318 tool calls per task with one action per step, 160.7 with several actions per call — OSWorld 2.0, 108 long tasks, Claude Opus 4.7

(XLANG Lab, 2026)

I-3 · NAVIGATION-ONLY STEPS

Most steps are navigation that needs no thinking, yet each one is a model call

- 66.7% of the steps in the median task are pure navigation — 151 WebVoyager tasks on live websites, three text agents driven by GPT-4o

(Wong et al., 2026)

I-4 · IDLING AND DEAD LOOPS

When an agent is stuck, every repeated step is billed and timed in full

- One element-locating loop repeated a step 18 times: 27 minutes and \$8.47 at list price without caching — GTA1 harness, OSWorld

(Abhyankar, Qi & Zhang, 2026)

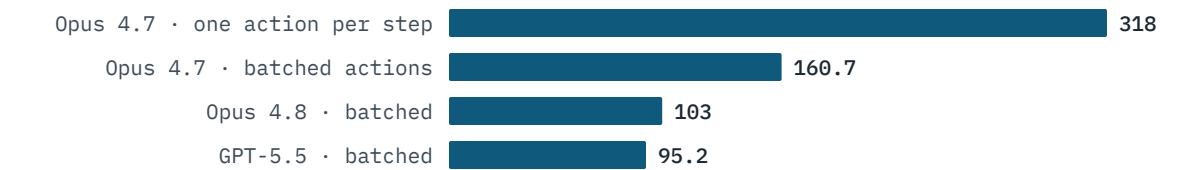
II · PLANNING, JUDGING, SAMPLING

Extra calls per step buy accuracy at a rising token cost per point

- 10 candidates per step instead of 1: 38.8% → 43.2% success for 96K → 920K tokens per task — gpt-oss-120b, 165 WebArena-Lite tasks

(Lee et al., 2026)

Steps per task on OSWorld 2.0, 108 tasks



XLANG Lab, 2026 · mean over tasks

III · Inside one call: first token, then decoding

from the two sums → here: the latency of one call, TTFT plus decoding → to: why the prompt keeps growing

A call waits in a queue, reads its uncached input, then writes its output one token at a time: $\ell_{ij} \approx \text{TTFT}_{ij} + n_{ij}^{\text{out}} \text{TPOT}_{ij}$. Queueing depends on the provider, reading on the prompt, writing on how much the model writes, thinking included.

III-1 · QUEUEING

The same request can wait far longer depending on the provider's load

- Identical requests take up to 69× longer depending on the time of day — 15 models, 5 providers

(Bian et al., 2025)

III-2 · PREFILL

For screenshot agents, reading the prompt dominates a call, and the prompt grows every step

- Planning, judging and reflection latency is “dominated by the prefill stage” — GTA1 and Agent S2 harnesses, 39 OSWorld tasks

(Abhyankar, Qi & Zhang, 2026)

III-3 · DECODING

Writing is the slowest segment per token, and thinking modes write far more tokens

- 224K vs 37K output tokens per task — Claude Opus 4.8 vs GPT-5.5 on the same 108 OSWorld 2.0 tasks
- Writing is 91–98.6% of model time — local 27–31B models with a warm cache

(XLANG Lab, 2026; Yuan et al., 2026)

HOW TTFT IS MEASURED

Published first-token times lump queueing, network and prefill together

- Time to first token is measured from the request being sent and “includes network latency”; output speed excludes the first chunk

(Artificial Analysis, 2026)

The prompt grows: every step re-reads the whole history

from one call → here: the prompt row of page 3, measured → to: the time outside the model

Each step appends its output and its result, so the tokens read over an attempt grow with N^2 ; caching makes re-reading cheaper and faster, not free. The square matters once N passes a few dozen steps.

HISTORY RE-SENT

Most computer-use agents send the whole history at every step

- “At each step, the prompt sent to the LLM includes the history of all previous steps”; later steps take up to 3× longer — GTA1 harness, 39 OSWorld tasks

(Abhyankar, Qi & Zhang, 2026)

SCREENSHOTS ARE THE BULK

One screenshot adds about a thousand tokens to every later prompt

- A 1280×720 screenshot is $46 \times 26 = 1,196$ tokens on Claude, one token per 28×28-pixel patch (calc.)

(Anthropic, 2026b)

CACHED, STILL PAID

A high cache hit rate lowers the price of re-reading, but the prefix stays the larger part of the bill

- 95.7% hit rate, yet prefix tokens are 59.5% of the list-price cost — 4,265 Claude Code / Codex sessions

(Zhu et al., 2026)

WHEN THE SQUARE BITES

Below a few dozen steps the first prompt dominates; long tasks are deep in the square

- $N^* = 2|H_{a,1}|/\bar{g} + 1$: a 6,000-token first prompt and 1,500 tokens added per step give $N^* = 9$ (calc., assumed values); OSWorld 2.0 tasks take about 318 steps

(XLANG Lab, 2026)

Per step, the pressure on prefill is the appended part, not the whole prompt: with a warm cache, “the relevant per-turn prefill pressure is better captured by incremental appends” (Yuan et al., 2026).

IV–V · The time outside the model, and how little of it overlaps

from the prompt → here: non-model time and concurrency → to: which term dominates

The environment is slow, and today's agents run almost everything in sequence, so T_{saving} is small and the attempt time is close to the full sum. Published “total times” differ in what they count, so the boundary has to be stated.

IV · NON-MODEL TIME

The environment is slow on its own, and the benchmarks add fixed sleeps on top

- OSWorld 2.0 sleeps 3 s after every action: × 318 steps ≈ 16 min of pure waiting per task (calc.)
- 6.6 s per browser action vs 4.7 s per model call, median — 151 WebVoyager live-site tasks, GPT-4o

(XLANG Lab, 2026; xlang-ai, 2026; Wong et al., 2026)

V · CONCURRENCY

Almost nothing overlaps, so fixing one term saves only its own share

- Concurrency within a turn: 1.15 — GitHub Copilot production telemetry

(Liu et al., 2026)

Where a Claude Code / Codex request’s 4.3 minutes go



Zhu et al., 2026 · 4,265 sessions, 43 developers

WHAT A “TOTAL TIME” COUNTS

Some published times include the person, others exclude them

- 80–92% of a coding session’s elapsed time is the person thinking between turns, outside any one attempt

(Liu et al., 2026; Zhu et al., 2026)

Which term dominates depends on the harness, not on the kind of agent

from the five terms → here: their shares, per kind of agent → to: where the money goes

All five classes exist in every agent; the heaviest term differs, and it flips when the harness changes.

	SCREENSHOT DESKTOP AGENT, MULTI-CALL HARNESS	TEXT WEB AGENT	CODING AGENT WITH A PROMPT CACHE
HEAVIEST	Reading × calls per step × steps. 87–97% of task time is planning, judging and reflection calls (GTA1, Agent S2; 39 OSWorld tasks) (Abhyankar, Qi & Zhang, 2026)	Browser execution and waiting. Per step, median: browser 6.6 s, model 4.7 s (151 WebVoyager live-site tasks, GPT-4o) (Wong et al., 2026)	Writing, or tool tails. Writing is 91–98.6% of model time (Yuan et al., 2026); in Claude Code / Codex requests tools take 59.8%, the model 41.0% (Zhu et al., 2026)
FLIPS WHEN	The harness makes one call per step: over 70% of the time is then the sandbox, because the benchmark sleeps 2–3 s after every action (Chang et al., 2026; xlang-ai, 2026; XLANG Lab, 2026)	The observation grows: 4.8× the input moved the model’s share of step time from 46.9% to 61.6% (Chang et al., 2026)	Load evicts the cache: latency up to 7.14× (Kang et al., 2026)
IN ONE LINE	Slow in reading: every step re-reads the growing screenshot history	Slow in the environment — until the observation grows, then slow in reading	Reading is cached away; slow in writing and in the tools

1 · The harness sets the dominant term — calls per step, observation size, fixed sleeps, cache on or off. The explanation “agents are slow because inference is slow” is accurate only for multi-call screenshot agents.

2 · N multiplies every term — one step fewer saves a whole step of time and money in all three kinds.

Where the money goes: six classes, one per term

from where the time goes → here: the terms of money per success, each with one measurement → to: per attempt vs per success

Money comes from tokens read and tokens written, each times a price, plus billed environment usage, which is usually small. Writing costs 5× reading; a cached read costs a small fraction of a normal read (DeepSeek-V4.1-Flash: \$0.006 vs \$0.30 per million tokens); fast mode costs 2× (Anthropic, 2026b; OpenAI, 2026; DeepSeek, 2026).

$$v(m, p) = \left[\mathbb{E} \left[\sum_i \sum_{j \leq J_i} \right] \right. \left. \sum_{\kappa} n_{ij}^{\kappa} \right] \left[c_{\kappa}(\mu_{ij}) \right] \left[+ x_{\text{env}} c_{\text{env}} \right] \left[\div R_m(p) \right]$$

3 · NUMBER OF CALLS

1, 2 · TOKENS BY CLASS

5 · PRICE PER CLASS

6 · ENVIRONMENT

4 · FAILURES, RETRIES

1 · TOKENS READ

Reading is the larger bill and grows with the square of the steps

- Even at a 95.7% cache hit rate, prefix tokens are still 59.5% of the bill — 4,265 Claude Code / Codex sessions

(Zhu et al., 2026)

2 · TOKENS WRITTEN

Writing is the dearer token, thinking included, but the smaller bill

- 31% of an uncached screenshot agent’s bill: \$2.43 counting output only vs \$7.87 counting all tokens — GTA1, 39 OSWorld tasks (calc.)

(Abhyankar, Qi & Zhang, 2026)

3 · NUMBER OF CALLS

The calls that make the agent slow also make it expensive

- 5–13× the calls of a single-call harness per step — GTA1: planners, retries and a judge (calc.)

(Abhyankar, Qi & Zhang, 2026)

4 · FAILURES AND RETRIES

Failed attempts are billed in full

- \$72.4 per attempt ÷ 20.6% completion ≈ \$351 per success — OSWorld 2.0, best agent (calc.)

(XLANG Lab, 2026)

5 · UNIT PRICE

The price list sets the constant; fast tiers double it

- Claude Opus 5.5 \$4 / \$20 vs GPT-6 Luna \$0.10 / \$0.50 per million tokens read / written — a 40× spread (calc.)

(Anthropic, 2026b; OpenAI, 2026)

6 · THE ENVIRONMENT MACHINE

Published costs are API bills only; a CPU environment adds little

- \$0.02–0.36 for a one-hour CPU environment: 0.3–4.6% of a \$7.87 bill (calc.; next page)

vendor price pages, read 2026-09-27

Four accounting conventions for one task, and the cost of the environment machine

from the six classes → here: per attempt vs per success, which convention a figure uses, and the environment → to: how time and money are linked

The same run’s cost differs by up to 30× across the four conventions in use, and none includes the environment machine.

CONVENTION	THE SAME RUN, COUNTED BOTH WAYS	GAP
OUTPUT ONLY VS ALL TOKENS	GTA1 on 39 OSWorld tasks, o3 list price, no cache: \$2.43 counting output only; \$7.87 counting all tokens (Abhyankar, Qi & Zhang, 2026)	3.2×
PER ATTEMPT VS PER SUCCESS	OSWorld 2.0, Claude Opus 4.8: \$72.4 per attempt at 20.6% completion → ≈ \$351 (calc.; a benchmark average) (XLANG Lab, 2026)	4.9×
UNCACHED VS CACHED	A cached read costs a fraction of a normal read (DeepSeek-V4.1-Flash: \$0.006 vs \$0.30 per million tokens; other vendors in A3) (DeepSeek, 2026)	4–50× on the read price only, by vendor (calc.)
STANDARD VS FAST MODE	Claude Opus 5.5 \$4 / \$20 → \$8 / \$40; OpenAI 2× on every listed model; up to 2.5× faster writing (vendor-stated), reading unchanged (Anthropic, 2026b; OpenAI, 2026)	2×

The three that apply to a whole bill compound to about 31× (3.2 × 4.9 × 2, calc.). Screenshot-agent measurements do not report their cache state.

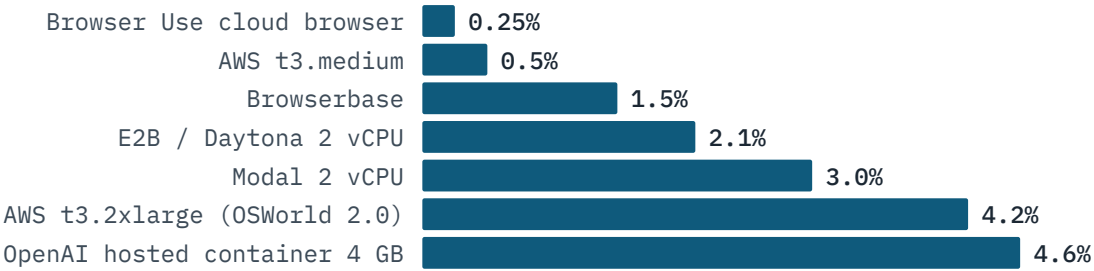
6 · THE ENVIRONMENT MACHINE, FROM LIST PRICES

For CPU environments the machine is a small addition; idle time, proxy traffic and GPUs change that

- An H100 sandbox at \$3.95 per hour is already half of a \$7.87 bill, and hourly billing runs while the agent waits — the median AgentSysBench session is active only 20% of its lifetime

vendor price pages, read 2026-09-27; Chang et al., 2026

A one-hour CPU environment as a share of a \$7.87 API bill (calc.)



vendor price pages, read 2026-09-27 · full price table in Appendix A3

How slow and expensive are linked: three kinds of relationship

from the two sides → here: what each term does to time and to money → to: the levers

Time and money share the token counts; queueing and environment time reach money only through billed environment usage — and through the cache, which expires while the agent waits.

TIME

≈

$\sum_i$

[

$\sum_j$

(

t_{ij}^{queue}

+

t_{ij}^{prefill}

+

n_{ij}^{out} TPOT $_{ij}$

)

+

E_i

]

−

T_{saving}

MONEY

=

$\sum_i$

$\sum_j$

$\sum_{\kappa}$

n_{ij}^{κ}

$c_{\kappa}(\mu_{ij})$

+

x_{env}

c_{env}

in both — same source

time only, or money only through billed environment usage

the price — money for speed or accuracy

t_{ij}^{queue} , t_{ij}^{prefill} the queueing and prefill parts of TTFIT_{ij} (our split; network and first-token time left out; Appendix A0)

time, money one attempt each; divide by $R_m(p)$ for per success

SAME SOURCE — IN BOTH EQUATIONS fix it, and time and money fall together Steps N, calls per step J_i, reading, writing, cache hits, failures — every one of them lives in the token counts (Abhyankar, Qi & Zhang, 2026; Anthropic, 2026b)	SLOW BUT NOT EXPENSIVE — NO TOKENS, ONLY MACHINE HOURS fix it, and only time falls Queueing, page loads, fixed sleeps, tool tails — none produces a token; the machine behind them is billed by the hour, usually under 5% of the API bill	MONEY FOR TIME OR FOR ACCURACY — OPPOSITE SIGNS paying for speed shortens only the writing segment; paying for accuracy costs more tokens per point as accuracy rises Fast mode: up to 2.5× faster writing (vendor-stated) at 2× the price, reading unchanged (Anthropic, 2026b; OpenAI, 2026; accuracy: Appendix A4)	TIME THAT COSTS MONEY — THE CACHE EXPIRES a slow step or a long pause can turn cheap cache reads into cache writes A cache entry lives 5 minutes or 1 hour from the start of the last request that read or wrote it; at most 12.8% of cost could be saved if the cache survived human pauses (Anthropic, 2026b; Zhu et al., 2026)
--	--	---	---

SOURCES · Abhyankar, Qi & Zhang, 2026 · Anthropic, 2026b · OpenAI, 2026 · Zhu et al., 2026 — the cause-by-cause table, with the convexity evidence, is Appendix A4

Appendix A4 ➤

The levers, and what is still unmeasured

from how time and money are linked → here: what fixing each lever buys, and the holes in the evidence → to: Part 2, what the literature has done with each lever

Seven levers follow from the formulas; each moves one term and carries a known risk. Part 2 asks how far the literature has pushed each. Right: gaps in the evidence, not claims of a research gap.

LEVER	TIME	MONEY	THE CATCH
Fewer steps N — several actions per call, skip navigation-only steps	yes	yes, the quadratic term	which steps need no thinking?
Read less $ H_{a,i} $, n^{unc} — trim history, shrink screenshots, cache the prefix	yes	yes	forgets; trimming invalidates the cache
Write less n^{out} — less thinking, smaller model	yes	yes	accuracy may fall
Fewer calls per step J_i — drop judging and reflection	yes	yes	fewer errors caught
Overlap the waiting T_{saving} — pre-load, parallel tools, page-ready events	yes	no	only where the environment dominates
Buy a fast or priority tier TPOT, c_κ	writing only	no — 2× more	reading unchanged; cache dropped
Stop retrying failures $R_m(p)$ — early stop, detect dead loops	yes	yes	stops some attempts that would have succeeded

NOT YET MEASURED

- **Read / write time split and cache hit rate for a frontier API model** — the only split is on local 27–31B models (Yuan et al., 2026); screenshot-agent measurements do not report cache state.
- **How much slower than a person** — no benchmark times agent and human on the same tasks; the closest, AXIS’s small user study, has a UI agent 1.69× slower than manual work on easy tasks (Lu et al., 2025).
- **Whether the benchmark’s fixed sleep sits inside “action” time** in OSWorld-Human — if so, “environment under 3.5%” understates the waiting.
- **How much slowness costs accuracy** — OSWorld 2.0 records the stale-screen failure mode without its share (XLANG Lab, 2026).

References · Part 1 (1 of 2)

author-year tags on the pages refer to these entries · published version where one exists, otherwise the arXiv number and the authors' institutions

1. Abhyankar, R., Qi, Q., & Zhang, Y. (2026). OSWorld-Human: Benchmarking the efficiency of computer-use agents. *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys 2026)*. arXiv:2506.16042. University of California, San Diego.
2. Anthropic. (2026a, May 13). *Best practices for computer and browser use with Claude*. <https://claude.com/blog/best-practices-for-computer-and-browser-use-with-claude>
3. Anthropic. (2026b). *Claude API pricing; Prompt caching; Fast mode; Vision* (developer documentation, read 28 September 2026). <https://platform.claude.com/docs/en/about-claude/pricing>; <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>; <https://platform.claude.com/docs/en/build-with-claude/fast-mode>; <https://platform.claude.com/docs/en/build-with-claude/vision>
4. Artificial Analysis. (2026). *Methodology; Performance benchmarking methodology* (read 28 September 2026). <https://artificialanalysis.ai/methodology>; <https://artificialanalysis.ai/methodology/performance-benchmarking> — third-party measurement definitions.
5. Bian, S., Yan, M., Jayarajan, A., Pekhimenko, G., & Venkataraman, S. (2025). What limits agentic systems efficiency? arXiv:2510.16276. University of Wisconsin–Madison; University of Toronto; NVIDIA.
6. Chang, C., Zhou, Y., Fu, K., An, D., Feng, T., Lu, H., Yao, S., Guo, P., Yu, Y., Shan, Y., Li, B., Yuan, B., & Wang, W. (2026). From LLM inference to agentic workloads: Characterization and implications for serving systems (AgentSysBench). arXiv:2608.15127. Hong Kong University of Science and Technology; Alibaba Group; ByteDance.
7. Chen, H. M., Guo, J., Luk, W., & Fan, H. (2026). AOSpec: Action and observation co-speculation for low-latency agent serving. arXiv:2608.00881. Imperial College London.
8. Chen, L., Zaharia, M., & Zou, J. (2024). FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*. arXiv:2305.05176. Stanford University.
9. DeepSeek. (2026). *Models & pricing* (API documentation, read 28 September 2026); *DeepSeek-V4.1-Flash release* (10 September 2026). https://api-docs.deepseek.com/quick_start/pricing; <https://api-docs.deepseek.com/news/news260910>
10. Erol, M. H., El, B., Suzgun, M., Yuksekogonul, M., & Zou, J. (2026). Cost-of-Pass: An economic framework for evaluating language models. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2504.13359v2. Stanford University.
11. Feng, G., Mao, H., Dutta, P., & Gonzalez, J. E. (2026). Concurrency without model changes: Future-based asynchronous function calling for LLMs (AsyncFC). arXiv:2605.15077. University of California, Berkeley.
12. Google. (2026). *Gemini Developer API pricing* (read 28 September 2026). <https://ai.google.dev/gemini-api/docs/pricing>
13. Kang, H., Li, Z., Yang, X., Xu, W., Chen, Y., Wang, J., Chen, B., Krishna, T., Xu, C., & Arora, S. (2026). ThunderAgent: A fast, simple, and program-aware agentic inference system. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, poster. arXiv:2602.13692. Georgia Institute of Technology; University of Illinois Urbana-Champaign; Carnegie Mellon University; Together AI.
14. Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2025). AI agents that matter. *Transactions on Machine Learning Research*. arXiv:2407.01502. Princeton University.
15. Kapoor, S., Stroebl, B., Kirgis, P., Nadgir, N., Siegel, Z. S., Wei, B., ... Narayanan, A. (2026). Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2510.11977. Princeton University et al.
16. Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., & Gholami, A. (2024). An LLM compiler for parallel function calling. *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. arXiv:2312.04511. University of California, Berkeley.
17. Le Sellier de Chezelles, T., Gasse, M., Lacoste, A., Caccia, M., Drouin, A., Boisvert, L., ... Chapados, N. (2025). The BrowserGym ecosystem for web agent research. *Transactions on Machine Learning Research*. arXiv:2412.05467. ServiceNow Research et al. (figure not re-verified; used only in Appendix A1)

References · Part 1 (2 of 2)

continued

18. Lee, N., Erdogan, L. E., John, C. J., Krishnapillai, S., Mahoney, M. W., Keutzer, K., & Gholami, A. (2026). Agentic test-time scaling for WebAgents. arXiv:2602.12276. University of California, Berkeley.
19. Li, Y., Ye, Q., Choubey, P. K., Zhang, J., & Wu, C.-S. (2026). Speculate with memory. arXiv:2607.12236. Salesforce Research.
20. Liu, B., Qiu, H., Goiri, Í., Fonseca, R., Bianchini, R., & Choukse, E. (2026). Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale. arXiv:2608.00101. University of Illinois Urbana-Champaign; Microsoft Azure Research.
21. Lu, J., Zhang, Z., Yang, F., Zhang, J., Wang, L., Du, C., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). AXIS: Efficient human-agent-computer interaction with API-first LLM-based agents. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7711–7743. <https://doi.org/10.18653/v1/2025.acl-long.381>. Microsoft.
22. Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., & Stoica, I. (2026). Agentix: An efficient serving engine for LLM agents as general programs. *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. University of California, Berkeley; Google DeepMind; Shanghai Jiao Tong University.
23. OpenAI. (2026). *API pricing; API changelog* (fast mode, 30 July 2026; Ultrafast, 13 August 2026) (read 28 September 2026). <https://developers.openai.com/api/docs/pricing>; <https://developers.openai.com/api/docs/changelog>
24. Vendor price pages used on pages 11–12 and in Appendix A3 (all read 27 September 2026): AWS, *Amazon EC2 T3 instances*; Browser Use, *Pricing* and *API v4: create browser session*; Browserbase, *Pricing* and *Billing plans*; Daytona, *Pricing* and *Billing*; E2B, *Pricing*; Modal, *Pricing* and *Sandbox resources*.
25. Winston, C., Wang, R. Y., Mirhoseini, A., & Kozyrakis, C. (2026). Agent JIT compilation for latency-optimizing web agent planning and scheduling. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, PMLR 306. arXiv:2605.21470. Stanford University.
26. Wong, M., Hsieh, K., Nath, S., & Netravali, R. (2026). Skim: Speculative execution for fast and efficient web agents. arXiv:2605.16565. Princeton University; Microsoft Research.
27. xAI. (2026). *Grok 4.7* model page, SpaceXAI Docs (read 28 September 2026). <https://docs.x.ai/developers/models/grok-4.7>
28. XLANG Lab. (2026). OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. arXiv:2606.29537. The University of Hong Kong (authored as “XLANG Lab and Collaborators”; the full author list is in the paper’s Appendix A).
29. xlang-ai. (2026). *OSWorld* [Source code], `desktop_env/desktop_env.py` and `run.py` at commit b138d348. <https://github.com/xlang-ai/OSWorld>
30. Xu, B., Xue, Z., Chen, D., Fu, C., Wu, C., Huang, C., ... Zhang, N. (2026). TokenPilot: Cache-efficient context management for LLM agents. arXiv:2606.17016. Zhejiang University; University of Electronic Science and Technology of China; Xidian University; HomologyAI.
31. Yuan, Y., Nayak, A., Kundu, S., & Talati, N. (2026). Agentic AI workload characteristics. arXiv:2605.26297v2 (21 September 2026). University of Illinois Urbana-Champaign; Gimlet Labs; Intel.
32. Zhang, C., Huang, H., Ni, C., Mu, J., Qin, S., He, S., ... Zhang, D. (2026). UFO2: The desktop AgentOS. *Transactions on Machine Learning Research*. arXiv:2504.14603. Microsoft.
33. Zhu, K., Jacob, M., Ma, C., Pan, Y., Wang, S., Krishnamurthy, A., & Kasikci, B. (2026). TraceLab: Characterizing coding agent workloads for LLM serving. arXiv:2606.30560. University of Washington.

A0 · The time of one attempt: each source’s own formula, and why ours differs

the derivation behind page 3 · verbatim / adapted / our distillation · calc. = our algebra

FORMULA (PAGES 2–3)	STATUS	THE SOURCE’S OWN FORMULA OR WORDS, IN ITS NOTATION	WHAT CHANGED, AND WHY
$T_{\text{attempt}} \triangleq t_{\text{end}} - t_{\text{start}} = \mathcal{D}(\mathcal{M} \cup \mathcal{E}) = \sum_{i=0}^N (D_i + E_i) - T_{\text{saving}}$	adapted	T_{saving} is “defined as the difference between the serialized baseline and the observed end-to-end asynchronous latency” (App. B.2) (Feng et al., 2026), combined with the sum below	one attempt, not one task (a task may take several); exact because the non-model intervals cover every instant; $T_{\text{saving}} \geq 0$ (calc.); exact with measured call times, approximate once $\ell \approx \text{TTFT} + n \text{ TPOT}$
$\sum_{i=0}^N (D_i + E_i)$	adapted	“At step i, the actor takes D_i time to generate action a_i , and the runtime takes T_i time to execute it ... yielding serial latency $\sum_i (D_i + T_i)$ ” (§3) (Chen et al., 2026)	$T_i \rightarrow E_i$ (T is kept for totals); D_i covers all calls of a step, E_i every non-model interval, both as summed lengths; $i = 0$ holds calls outside any step; AOSpec’s a_i is an action, not our agent index; preferred to (Kim et al., 2024), whose N counts the function calls of one plan
$D_i = \sum_j \ell_{ij}$	our distillation	No source sums the calls of a step. GTA1 makes “4 parallel calls” per planning step and “between 4 and 12 planning calls” per judging call (Abhyankar, Qi & Zhang, 2026)	parallel calls add here; their overlap is credited to T_{saving} ; the letter ℓ as in (Li et al., 2026)
$\ell_{ij} \approx \text{TTFT}_{ij} + n_{ij}^{\text{out}} \text{TPOT}_{ij}$	adapted	Total Response Time = Time to First Token + 100 / Output Speed (Artificial Analysis, 2026)	100 $\rightarrow$ the call’s own output tokens, thinking included; 1/speed $\rightarrow$ TPOT; $\approx$ because output speed excludes the first chunk; TTFT runs to the first token of any kind, reasoning included
$\text{TTFT}_{ij} \approx t^{\text{queue}} + t^{\text{net}} + t^{\text{prefill}} (n^{\text{unc}} + n^{\text{w}}; n^{\text{hit}}) + t^{\text{first}}$	our distillation	No source writes TTFT as a sum. A program’s latency “comprises three components”, the first the “queuing time” of its calls (Luo et al., 2026); TTFT “includes network latency” (Artificial Analysis, 2026)	the split used on page 13; t^{net} : network time, t^{first} : first decode step; the shape of t^{prefill} is not given by any verified source

A0 · Time hidden by concurrency, and the growing prompt

the derivation behind page 3 · verbatim / adapted / our distillation · calc. = our algebra

FORMULA (PAGE 3)	STATUS	THE SOURCE’S OWN FORMULA OR WORDS, IN ITS NOTATION	WHAT CHANGED, AND WHY
$T_{\text{saving}} \triangleq \mathcal{S}(\mathcal{M}) + \mathcal{S}(\mathcal{E}) - \mathcal{D}(\mathcal{M} \cup \mathcal{E})$	adapted	the same formula, continued $= \Delta_{F \parallel F} + \Delta_{D \parallel E}$, with $\mathcal{M}, \mathcal{E}$ “the sequence of time intervals for model decoding and function execution” (App. B.2) (Feng et al., 2026)	intervals widened to whole calls and all non-model time, so values are not comparable with AsyncFC’s; the split becomes (calc.) $[\mathcal{S}(\mathcal{M}) - \mathcal{D}(\mathcal{M})] + [\mathcal{S}(\mathcal{E}) - \mathcal{D}(\mathcal{E})] + [\mathcal{D}(\mathcal{M}) + \mathcal{D}(\mathcal{E}) - \mathcal{D}(\mathcal{M} \cup \mathcal{E})]$, each ≥ 0 ; it can include extra work that concurrency causes, so it is not the saving against a separate serial run (speed-ups use $T_{\text{seq}}/T_{\text{attempt}}$, Part 2)
$ H_{a,i+1} = H_{a,i} + \Phi(z_{a,i}) + o_{a,i} $	adapted	$H_{a,i+1} = H_{a,i} \parallel \Phi(\theta_{a,i}, m_{a,i}, u_{a,i}) \parallel o_{a,i}$, $C_{a,i} = H_{a,i} $ (Eqs. 1–3, v2 §II-C) (Yuan et al., 2026)	lengths of the concatenation; $z = (\theta, m, u)$ (thinking, message, tool-call tokens) substituted, because Yuan’s m would clash with the design m and its $C_{a,i}$ with the cost $C_m(p)$; exact with one call per step and full history; in general add $h_i^+ - h_i^-$ (other calls’ output; compaction)
$\sum_{i=1}^N H_{a,i} = N H_{a,1} + \frac{N(N-1)}{2} \bar{g}$	our distillation (calc.)	no source writes the sum; “at each step, the prompt sent to the LLM includes the history of all previous steps” (Abhyankar, Qi & Zhang, 2026)	$g_i = \Phi(z_{a,i}) + o_{a,i} $, $\bar{g} = \sum_i (N-i) g_i / \frac{N(N-1)}{2}$ makes it exact; over a random step count $\mathbb{E}[\sum H] \approx \mathbb{E}[N] H_{a,1} + \frac{\bar{g}}{2} (\mathbb{E}[N]^2 + \text{Var}[N] - \mathbb{E}[N])$; crossover $N^* = 2 H_{a,1} /\bar{g} + 1$

A0 · Money, per success, and the goal

the derivation behind page 4 · verbatim / adapted / our distillation · calc. = our algebra

FORMULA (PAGE 4)	STATUS	THE SOURCE’S OWN FORMULA OR WORDS, IN ITS NOTATION	WHAT CHANGED, AND WHY
$c_m(p) = \sum_{i,j} \sum_{\kappa} n_{ij}^{\kappa} c_{\kappa}(\mu_{ij}) + x_{\text{env}} c_{\text{env}}$	adapted	$c_m(p) = n_{\text{in}}(m, p) c_{\text{in}}(m) + n_{\text{out}}(m, p) c_{\text{out}}(m)$ (Eq. 13): input and output tokens times their prices; other components go into $C_m(p) = w^{\top} x_m(p)$, w the unit prices and $x_m(p)$ the quantities per attempt (App. D.1) (Erol et al., 2026)	input split into the billed classes (cache hit, 5-min and 1-h cache write, uncached); prices inside the sum because one step can mix models; environment usage added as App. D.1 allows; the class index is κ because x is Cost-of-Pass’s quantity. Closest written three-class form: (Xu et al., 2026)
$v(m, p) = C_m(p) / R_m(p)$	verbatim	“the expected number of attempts to obtain the first correct solution is $1 / R_m(p)$ ”, assuming independent trials (§2.2, Eq. 2) (Erol et al., 2026)	none; assumes unlimited independent retries and the verifier’s time and cost in every attempt; with at most K retries the ratio is still C/R (calc.); with cache-warm retries $C_m^{(1)}(p) + (1 / R_m(p) - 1) C_m^{(2+)}(p)$ (calc.)
$\mathbb{E}[T_{\text{attempt}}] / R_m(p)$	adapted	“alternative units per attempt (FLOPs, time, latency, energy) may matter more than dollar cost” (App. D.1) (Erol et al., 2026)	the same derivation in seconds; serial retries only (parallel votes cost “roughly the latency of a single vote”); correlated failures break independence
$\text{Pareto}_m(\cdot, \cdot)$ s. t. $R_m(p) \geq R_0$	our distillation	$V_p(\mathcal{M}) = \min_{m \in \mathcal{M}} v(m, p)$ (Eq. 3), with a human expert as fallback, $\min(V_p(\mathcal{M}), v(\text{expert}, p))$ (Eqs. 4–5) (Erol et al., 2026); “the new goal of jointly optimizing the two metrics” (arXiv v1) (Kapoor et al., 2025); budgeted dual: maximise quality with expected cost within a budget (Chen, Zaharia & Zou, 2024)	$\text{min} \rightarrow \text{Pareto set}$, because seconds and dollars are not one scalar; the human is the reference point; the floor R_0 follows App. C.8’s remedy of leaving unreliable systems off the frontier; over a task mix $\sum_p C_m(p) / \sum_p R_m(p)$, unlike Eq. 8’s mean of V_p

A0 · Every symbol, its unit, and where it comes from

symbols follow their source wherever the source has one; ours are marked

SYMBOL	MEANING	UNIT	ORIGIN
i, N, j, J_i, a	step (i = 0: calls outside any step); steps; model call; calls in step i; agent	—; steps; —; calls; —	i as in AOSpec and Yuan et al.; j, J_i ours (calls ≠ steps); a from Yuan et al.
$D_i, E_i, T_{\text{attempt}}, T_{\text{saving}}, T_{\text{seq}}$	model time; non-model time of step i; wall-clock of one attempt; time hidden by concurrency; a separate serial run	s	D from AOSpec, widened; E ours (letter from AsyncFC's $\mathcal{E}$, AsyncLM's E); T_saving AsyncFC, widened; T_seq from speculation papers
$\mathcal{S}, \mathcal{D}, \mathcal{M}, \mathcal{E}, \triangle$	summed length; length of a union; model-call and non-model intervals; defined as	s	AsyncFC (intervals widened)
$\ell_{ij}, \text{TTFT}_{ij}, \text{TPOT}_{ij}$	call latency; time to first token; time per output token	s; s; s/token	ℓ as in (Li et al., 2026); TTFT (Artificial Analysis, 2026); TPOT as in (Chen et al., 2026)
$t^{\text{queue}}, t^{\text{net}}, t^{\text{prefill}}, t^{\text{first}}$	parts of TTFT: queueing, network, prefill of uncached input, first decode step	s	ours (A0 · 1/5)
$n^{\text{in}}, n^{\text{hit}}, n^{\text{w}}, n^{\text{w5m}}, n^{\text{w1h}}, n^{\text{unc}}, n^{\text{out}}$	tokens: all input (= hit + w + unc); cache read; cache write (= w5m + w1h); uncached input; output	tokens	classes from Anthropic's usage fields; hit as in TokenPilot, unc as in SpeedRunner; n from Cost-of-Pass
$\kappa, K, \mu_{ij}, c_{\kappa}(\mu)$	billing class; the set {hit, w5m, w1h, unc, out}; serving model; price per token of class κ on μ	—; —; —; \$/token	ours; c as in Cost-of-Pass
$x_{\text{env}}, c_{\text{env}}, w, x_m(p)$	billed environment usage; its price; price vector; quantity vector per attempt	h; \$/h; \$/unit; units	Cost-of-Pass App. D.1
$H_{a,i}, \Phi, z_{a,i}, o_{a,i}, g_i, \bar{g}, N^*$	context before step i of agent a; chat template; the step's output; its tool result; tokens added; weighted mean; crossover	tokens; —; tokens; tokens; tokens; tokens; steps	Yuan et al.; g, $\bar{g}$, N* ours (calc.)
$m, p, c_m(p), C_m(p), R_m(p), v(m,p), R_0, \mathbb{E}, \text{Var}$	design (model + harness); task; dollars of one attempt; expected; success probability; dollars per success; success floor; expectation; variance	—; —; \$; \$; prob.; \$; prob.	Cost-of-Pass; R_0 ours

A0 · What a published number covers: map it onto the terms before comparing

the measurement boundary · a figure is comparable only with what it includes stated

KIND OF FIGURE	WHAT IT INCLUDES	WHAT IT LEAVES OUT
first-token time, output speed (Artificial Analysis, 2026)	one synthetic call: queueing, network and prefill lumped in TTFT; speed after the first chunk	the step, the tools, the environment
request response time (Zhu et al., 2026)	model generation plus tool execution	the person thinking between turns
stopwatch time per task (Abhyankar, Qi & Zhang, 2026)	the whole attempt, T_{attempt}	— (39 tasks, one run each)
time saved (Feng et al., 2026)	decode and function-execution intervals of the concurrent trace	queueing, prefill, harness gaps
cost with two classes (input, output) (Erol et al., 2026)	all input at the base price	cache reads (overstates cost) and cache writes (understates it)
cost with three classes (hit, miss, output) (Xu et al., 2026)	cache reads at their price	cache writes; environment usage
cache hit rate	denominators differ: cached / cacheable input (Artificial Analysis, 2026), / all input (Xu et al., 2026), per incoming call (Luo et al., 2026)	ours: $n^{\text{hit}}/n^{\text{in}}$
almost every published figure	per attempt	per success: divide by $R_m(p)$; lossless speed papers report no task success

A1 · Slowness, every cause — I. steps and II. calls per step

the complete table behind pages 5–6 · three kinds of agent: screenshot desktop agent, text web agent, coding agent with a prompt cache

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
I-1	The task is long	Hour-scale tasks take hundreds of steps	OSWorld 2.0, 108 long tasks: Claude Opus 4.7 with maximum thinking, one action per step, 500-step cap: 318 tool calls per task on average, i.e. 318 steps; with several actions per call, Claude Opus 4.8 takes 103 steps (481.8 calls) and GPT-5.5 95.2 steps (149.8 calls)	(XLANG Lab, 2026)	screenshot, text
I-2	One action per step	Every click is a whole step: one observation, one model call, one wait	OSWorld 2.0: with one action per step Claude Opus 4.7 needs 318 steps; with batched actions the same model needs 160.7 and Claude Opus 4.8 needs 103	(XLANG Lab, 2026)	screenshot, text
I-3	Navigation-only steps still call the model	Paging, scrolling and opening the next page need no thinking but cost a call each	Skim’s profile of three text web agents (Browser-Use, AgentOccam, WebVoyager; GPT-4o) on 151 WebVoyager live-site tasks: 66.7% of the steps in the median task are pure navigation	(Wong et al., 2026)	text
I-4	Idling and dead loops	Element not found, repeated retries; steps rise without progress	OSWorld-Human’s records of the GTA1 harness (o3 plans and judges, GTA1-7B locates elements) on 39 OSWorld desktop tasks: in failed tasks that exceeded 50 steps, 66% of steps were wasted repeats; one element-locating loop repeated the same step 18 times — 27 minutes and \$8.47 at list price without caching	(Abhyankar, Qi & Zhang, 2026)	screenshot
I-5	Acting on a stale screen	The screenshot is taken before the page changes; the agent clicks on the old layout and must repair	OSWorld 2.0 records “stale interface state” as a failure mode; no share reported	(XLANG Lab, 2026)	screenshot
II-1	Multi-call harness	One step = plan + judge + reflect, several calls in series	OSWorld-Human, two harnesses: GTA1 runs 4 parallel planners per step, retries up to 3 rounds, then one judging call picks — 4–12 planning calls per judging call; Agent S2 adds a reflection call per step, so planning is 53% and reflection 34% of task time	(Abhyankar, Qi & Zhang, 2026)	screenshot
II-2	Sampling for accuracy	5–20 candidates per step, then pick one	Same model and harness (gpt-oss-120b, ReAct) on 165 WebArena-Lite tasks: 1 → 10 candidates per step raises success from 38.8% to 43.2% and tokens per task from 96K to 920K — 9.6× the tokens for 4.4 points	(Lee et al., 2026)	text
II-3	Coding agents call repeatedly within a turn	The model is called back and forth inside one user turn	GitHub Copilot coding agent production telemetry (first week of June 2026, 13.5 million sessions): 6.6 model calls per user turn on average	(Liu et al., 2026)	coding

A1 · Slowness, every cause — III. each call is slow

the complete table behind pages 7–8

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
III-1	Queueing	The request waits at the provider; unbilled, but counted as model time	Client-side measurement of 15 models across 5 providers: requests of the same length differ in latency by up to 69× depending on when they are sent	(Bian et al., 2025)	all
III-2	Reading (the prompt is re-processed every call)	Every call re-reads the whole history; more screenshots, slower	OSWorld-Human, multi-call harnesses: later steps up to 3× slower because the prompt at step i holds the $i - 1$ earlier screenshots — “dominated by prefill”. One screenshot is 1,000–1,800 tokens. AgentSysBench’s WebArena agent (Kimi-K2.6): observation switched from a single format to accessibility tree + HTML + screenshot → input 4.8×, model share of time 46.9% → 61.6%. Under load the cache is evicted and re-read: SWE-Agent / OpenHands running GLM-4.6 on 8 H100s for SWE-bench Lite, request latency up to 7.14× as concurrency rises	(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026a; Chang et al., 2026; Kang et al., 2026)	screenshot; text with rich observations; coding under load
III-3	Writing (one token at a time)	Token-by-token generation, strictly serial	Locally served ReAct agents (Qwen3.6-27B / Gemma4-31B, vLLM, 2 H100s) on five benchmarks: with the cache warm, generation is 91–98.6% of model time. Windows desktop agent UFO2 (GPT-4o / o1 API): about 10 s per model call, the largest item per step in every configuration	(Yuan et al., 2026; Zhang et al., 2026)	coding; single-call screenshot
III-4	Thinking modes and larger models	Thinking writes more tokens; larger models are slower per token	OSWorld 2.0, same 108 tasks: Claude Opus 4.8 writes 224K output tokens per task, GPT-5.5 37K. Holistic Agent Leaderboard, 9 benchmarks, 21,730 runs: raising reasoning effort lowered accuracy in 21 of 36 pairs	(XLANG Lab, 2026; Kapoor et al., 2026)	all

A1 · Slowness, every cause — IV. environment, V. serial

the complete table behind page 9 (non-model time and concurrency)

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
IV-1	Producing the observation	Extracting the accessibility tree or detecting elements takes time	OSWorld desktop applications: 3–26 s to generate one accessibility tree; the screenshot itself is under 2% of task time. UFO2: OmniParser element detection adds about 1 s per step	(Abhyankar, Qi & Zhang, 2026; Zhang et al., 2026)	screenshot
IV-2	Browser execution and page load	After a click the agent waits for the page	151 WebVoyager live-site tasks: per step, median browser action 6.6 s vs model call 4.7 s. WebArena with GenericAgent + Claude 3.5 Sonnet: 7.6 s of a 12.2 s step in the browser (this figure was not re-verified)	(Wong et al., 2026; Le Sellier de Chezelles et al., 2025)	text
IV-3	Fixed sleeps	The agent cannot tell when the page is ready, so it sleeps a fixed number of seconds	OSWorld’s environment code sleeps 2 s after every action; OSWorld 2.0 prescribes 3 s, so 318 steps are about 16 minutes of pure waiting (calc.). AgentSysBench: a single-call GUI agent (ReAct, Kimi-K2.6 API) spends over 70% of its OSWorld execution time in the desktop sandbox	(xlang-ai, 2026; XLANG Lab, 2026; Chang et al., 2026)	screenshot (caused by the benchmark harness)
IV-4	Tool tails	Tests and builds take minutes	TraceLab, 4,265 Claude Code / Codex sessions from 43 developers (September 2025 – June 2026): tool calls over 1 minute are 4% of calls but 85% of tool time; a request averages 4.3 minutes — tools 2.5 (59.8%), model 1.7 (41.0%)	(Zhu et al., 2026)	coding
V	Serial structure	The four stages never overlap, so task time is the sum of every term, not the largest one; fixing one term saves only its share	Concurrency within a turn is 1.15 in Copilot telemetry. Separately, 80–92% of a coding session’s elapsed time is the human thinking between turns — strip it before reading any total	(Liu et al., 2026; Zhu et al., 2026)	all

A2 · Where the arrows land, with every measurement condition

the complete table behind page 10

	SCREENSHOT DESKTOP AGENT, MULTI-CALL HARNESS	TEXT WEB AGENT	CODING AGENT WITH A PROMPT CACHE
heaviest term	Reading × calls per step × steps. OSWorld-Human timed two multi-call harnesses (GTA1, Agent S2) step by step on 39 OSWorld desktop tasks: 87–97% of task time in planning, judging and reflection calls; screenshots and actions together under 3.5% (Abhyankar, Qi & Zhang, 2026)	Browser execution plus waiting. Skim measured three text web agents (GPT-4o) on 151 WebVoyager live-site tasks: per step, median browser action 6.6 s and model call 4.7 s (Wong et al., 2026)	Writing, or tool tails, depending on the tools. With the context cache on, token-by-token generation is 91–98.6% of model time (locally served models) (Yuan et al., 2026); in Claude Code / Codex records a request spends 59.8% in tools and 41.0% in the model (Zhu et al., 2026); in GitHub Copilot, whose tools are light, agent time is 13.7% model and 2% tools (Liu et al., 2026)
second	Writing: with thinking on, more reasoning tokens per step, and generation is one token at a time	Reading, once the observation grows: AgentSysBench switched a WebArena agent from one observation format to accessibility tree + HTML + screenshot — input 4.8×, model share 46.9% → 61.6% (Chang et al., 2026); the Browser-Use harness spends 73% of its latency in model calls (Winston et al., 2026)	Reading, when load evicts the cache and the whole context is re-read: ThunderAgent measured request latency up to 7.14× under cache thrash (Kang et al., 2026)
negligible	Producing the observation and executing the action: under 3.5% together (Abhyankar, Qi & Zhang, 2026)	Nothing: model and browser are close; which is larger depends on observation size and model speed	Navigation-only steps: a coding agent has no pages to turn
counter-example	A single-call agent is different: AgentSysBench’s one-call GUI agent (Kimi-K2.6) spends over 70% of its OSWorld time in the desktop sandbox (Chang et al., 2026), because the harness sleeps after every action — 2 s in OSWorld’s code (xlang-ai, 2026), 3 s in OSWorld 2.0 (XLANG Lab, 2026)	Browser-Use: 73% of latency in model calls, not the browser (Winston et al., 2026)	Two production records point opposite ways: heavy tools in Claude Code / Codex (tools 59.8% > model 41.0%) (Zhu et al., 2026); light tools in Copilot (model 13.7% > tools 2%) (Liu et al., 2026). The difference is the kind of tool (tests and builds vs file reads), not a contradiction
in one line	Slow in reading: every step re-reads the screenshot history, more as the task goes on	Slow in the environment (browser and page waits); once the observation grows, slow in reading again	Reading is cached away; slow in writing and in the tools

A3 · Cost, every cause — 1. tokens read and 2. tokens written

the complete table behind pages 8 and 11 · prices are list prices from the vendors’ own pages, September 2026

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE
1-1	History re-sent	Step i reads everything from the $i - 1$ earlier steps; total reading $\approx N^2$	“Cost grows quadratically with the number of steps” — OSWorld-Human, real runs	(Abhyankar, Qi & Zhang, 2026)
1-2	Large screenshots	1,000–1,800 tokens per image; 100 images fill a 200K context	Anthropic’s engineering guidance for computer and browser use	(Anthropic, 2026a)
1-3	Rich observation formats	Accessibility tree + HTML + screenshot together: 4.8× the input	AgentSysBench, WebArena agent, Kimi-K2.6	(Chang et al., 2026)
1-4	Sampling	Several candidates per step, mostly read tokens	gpt-oss-120b + ReAct on 165 WebArena-Lite tasks: 1 → 10 candidates per step, 96K → 920K tokens per task	(Lee et al., 2026)
1-5	The cache is not a cure	However high the hit rate, a long history keeps reading dominant; a miss re-reads everything	TraceLab, 4,265 Claude Code / Codex sessions: prefix-cache hit rate 95.7%, prefix tokens still 59.5% of list-price cost; on a miss the re-read is 3.8× the genuinely new content	(Zhu et al., 2026)
2-1	Thinking modes	Thinking tokens are billed as output; when they cannot be switched off they are a fixed overhead	OSWorld 2.0, same tasks: Claude Opus 4.8 224K output tokens per task, GPT-5.5 37K; Claude Opus 5.5: “thinking cannot be disabled and is billed as output”	(XLANG Lab, 2026; Anthropic, 2026b)
2-2	But writing is not always the larger bill	In an uncached screenshot agent, output was 31% of the total	OSWorld-Human, GTA1: the paper’s \$2.43 counts output only; Table 3’s input and output give \$7.87	(Abhyankar, Qi & Zhang, 2026)

A3 · Cost, every cause — 3. calls, 4. failures, 5. unit price

the complete table behind page 11 (calls, failures, unit price)

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE
3	Number of calls	$N \cdot J_i$ scales both token bills	GTA1: 4 parallel planners per step, up to 3 retry rounds, one judging call → 4–12 planning calls per judging call, i.e. 5–13× the calls of a single-call harness (calc.). Agentic test-time scaling: 5–20 candidates per step multiply the calls by the candidate count. The same extra calls cost time and money at once	(Abhyankar, Qi & Zhang, 2026; Lee et al., 2026)
4-1	Failed attempts are billed	Cost per success = cost per attempt ÷ success rate	OSWorld 2.0: the best agent (Claude Opus 4.8) costs about \$72.4 per attempt at 20.6% completion → about \$351 per success (calc.). An accounting conversion, not the real price of retrying one task until it succeeds	(XLANG Lab, 2026)
4-2	Idle steps are billed in full	Every step of a dead loop is a full read and write	OSWorld-Human, GTA1 on 39 OSWorld tasks: one element-locating loop repeated the same step 18 times — 27 minutes, \$8.47 at list price without caching; in failed tasks over 50 steps, 66% of steps were such repeats	(Abhyankar, Qi & Zhang, 2026)
4-3	Evaluation itself is too expensive to repeat	Single runs, no repeats	Holistic Agent Leaderboard: 9 benchmarks, 21,730 runs, one run per configuration, about \$40,000 in total; Claude Opus 4.1 not run on Online-Mind2Web because the estimate was \$20,000	(Kapoor et al., 2026)
5-1	Model tier	Claude Opus 5.5 \$4 / \$20 vs GPT-6 Luna \$0.10 / \$0.50 per million tokens read / written	Vendor price pages, 27 September 2026	(Anthropic, 2026b; OpenAI, 2026)
5-2	Fast mode	2× on both vendors; switching speed tiers invalidates the cache	Anthropic: Claude Opus 5.5 \$8 / \$40, up to 2.5× output speed (vendor-stated), first-token wait unchanged. OpenAI: 2× on all listed models; “up to 2.5×” stated only for GPT-5.6 Sol	(Anthropic, 2026b; OpenAI, 2026)
5-3	Cache write and read	Latest model of each vendor, per million tokens, normal read → cached read (ratios calc.): DeepSeek-V4.1-Flash \$0.30 → \$0.006 (0.02×; peak rate, both halve off-peak) Claude Opus 5.5 \$4 → \$0.20 (0.05×; other Claude models 0.1×, Fable 5.1 0.025×; cache writes 1.25× for 5 min, 2× for 1 hour) GPT-6 Sol \$2 → \$0.20 and GPT-6 Luna \$0.10 → \$0.01 (0.1×; cache writes 1.25×) Gemini 3.8 Flash \$0.75 → \$0.075 (0.1×; plus \$0.50 per million tokens per hour of storage; introductory to 31 Dec 2026) Grok 4.7 \$2 → \$0.50 (0.25×)	Vendor price pages, read 28 September 2026	(DeepSeek, 2026; Anthropic, 2026b; OpenAI, 2026; Google, 2026; xAI, 2026)
5-4	Long-context surcharge	OpenAI: 2× the read price above 272K tokens	Vendor price page	(OpenAI, 2026)

A3 · Cost — 6. the environment machine, and the four conventions

the complete tables behind page 12 · vendor price pages read 2026-09-27 · per-task amounts and shares are our own arithmetic (calc.)

ENVIRONMENT	PRICE (VENDOR PAGE)	1-HOUR TASK	15-MIN TASK	SHARE OF A \$7.87 API BILL	SHARE OF \$72.4
Browser Use cloud browser	\$0.02 per browser-hour, billed by the minute, 1-minute minimum; traffic extra (residential proxy \$5/GB, direct \$0.20/GB); sessions capped at 240 minutes	\$0.020	\$0.005	0.25%	0.03%
AWS t3.medium (2 vCPU / 4 GiB, Linux, us-east-1)	\$0.0418 per hour on the T3 product page; another AWS page says \$0.0416 (0.5% apart)	\$0.042	\$0.010	0.5%	0.06%
Browserbase cloud browser	\$20 per month for 100 hours, then \$0.12 per hour; \$99 per month for 500 hours, then \$0.10; billed by the minute	\$0.12	\$0.03	1.5%	0.17%
E2B / Daytona sandbox (2 vCPU / 4 GiB)	E2B \$0.000014 per vCPU-second + \$0.0000045 per GiB-second; Daytona \$0.0504 per vCPU-hour + \$0.0162 per GiB-hour — both \$0.1656 per hour	\$0.166	\$0.041	2.1%	0.23%
Modal sandbox (1 physical core = 2 vCPU, 4 GiB)	\$0.00003942 per core-second + \$0.00000667 per GiB-second → \$0.238 per hour; billed on the larger of requested and used	\$0.238	\$0.060	3.0%	0.33%
AWS t3.2xlarge (8 vCPU / 32 GiB, OSWorld 2.0's default instance)	\$0.3341 per hour	\$0.334	\$0.084	4.2%	0.46%
OpenAI hosted container (Hosted Shell / Code Interpreter, 4 GB)	\$0.12 per 20-minute session → \$0.36 per hour; 1 GB \$0.03, 16 GB \$0.48, 64 GB \$1.92 per 20 minutes	\$0.36	\$0.12	4.6%	0.50%

The four accounting conventions (page 12), in one line each: output only vs all tokens — 3.2× (OSWorld-Human, \$2.43 vs \$7.87); per attempt vs per success — 4.9× (OSWorld 2.0, \$72.4 ÷ 20.6%); uncached vs cached — 4–50× on the read price by vendor (calc.), far less on the bill (TraceLab: 95.7% hits, prefix still 59.5% of cost); standard vs fast mode — 2× (both vendors), buying only writing speed.

A4 · Each cause’s effect on time and on money

the complete table behind pages 13–14

CAUSE	EFFECT ON TIME	EFFECT ON MONEY	RELATIONSHIP
Many steps (N)	Roughly linear: one more step is one more call and one more wait (later steps somewhat slower, up to 3×)	Write tokens linear in N ; read tokens quadratic, because every step re-reads the whole history	same source — double the steps: about 2× the time, about 4× the read bill (uncached)
Many calls per step (J_i)	linear	linear	same source
Reading a lot (history, screenshots, rich observations)	Reading time, longer every step	Read-token bill	same source — the same tokens cost time and money
Writing a lot (thinking)	Generation time, roughly 12 ms per token	Write-token bill at 5× the read price	same source , and the dearest time: writing is the slowest and the most expensive token
Cache hits	Saves reading time	Read price 0.02–0.25× of a normal read, by vendor (A3)	same source , but fragile: editing the history or switching to fast mode invalidates it
Queueing	slow	free	slow but not expensive ; escaping it means a fast or priority tier at 2×
Environment waits (page loads, fixed sleeps)	slow	No API cost; machine time by the hour — CPU sandboxes \$0.02–0.36 per hour, usually under 5% of the API bill	slow but not expensive
Tool tails (tests, builds)	slow	No API cost	slow but not expensive
Failures, idling, retries	slow	expensive	same source , and it turns “per attempt” into “per success”
Fast mode	Writing up to 2.5× faster (vendor-stated); reading unchanged	2× the price	money for time
Bigger models, more thinking	slower	dearer	money and time for accuracy , convex: marginal tokens per point 101K → 575K on one model; 6× / 9× / 9.6× tokens for a few points; more thinking not always more accurate (Lee et al., 2026; XLANG Lab, 2026; Kapoor et al., 2026)
Smaller models	faster	cheaper	but if accuracy falls, steps and retries rise — possibly slower and dearer again (inference, no direct evidence)

A5 · The gaps in the evidence, and what each one blocks

the complete table behind page 14

GAP	WHAT EXISTS TODAY	CONSEQUENCE FOR WHAT CAN BE CLAIMED
Read / write time split and cache hit rate for a frontier API model	No measurement reports both; the only read / write split is on locally served 27–31B models (Yuan et al., 2026); screenshot-agent measurements do not report cache state	How much of “87–97% in the model” is reading, how much writing, and what remains once the cache is on — unknown
How much slower an agent is than a person	None of 17 benchmarks measures agent and human elapsed time on the same tasks; the closest is AXIS’s user study, in which a UI agent was 1.69× slower than manual work on easy tasks (small sample) (Lu et al., 2025)	“X times slower than a person” cannot go on a slide
Environment dollars	Benchmarks report instance types; AgentSysBench reports shares; products publish nothing. Computable from instance type × duration × list price: CPU environments usually 0.3–5% of the API bill (page 12)	Every benchmark cost is an API bill; the omission is small for CPU environments, not for GPU sandboxes or long idle sessions
Whether OSWorld-Human’s “action” includes the 2-s sleep after each step	Not stated (xlang-ai, 2026); if it does, “actions under 2%” implies more than 100 s per step (calc.)	“Environment under 3.5%” may understate the waiting
The rebound from a smaller model	No direct evidence on how many extra steps and retries a drop in accuracy causes	The last row of the relationship table stays marked “inference”
How much slowness hurts accuracy	Only OSWorld 2.0 records the stale-state failure mode, without its share of failures (XLANG Lab, 2026)	“It exists” can be said; “how large” cannot